

Assignment 2 - Deep Learning for Visual Computing

Milica Aleksic 12424821, Vidak Grujic 12332263

June 2025

1 Introduction

In this report, we compare performance of different models on image segmentation tasks where we used OxfordIIITPet and Cityscapes datasets. The models include PyTorch's pre-built FCN-ResNet50 and the transformer-based SegFormer, which was implemented as part of the assignment. The main goal was to understand how the architecture design and training strategies affect the segmentation performance. As strategies used, we consider pre-trained weights, freezing parts of the network and fine-tuning. We tested training from scratch, also used transfer learning with encoder-only pretraining. We also evaluated how changes like learning rate and weight freezing can impact the result. The evaluation is based on train and validation mIoU and loss curves. This allows us to examine which combination setup generalizes the best and how different choices influence the performance.

2 Theoretical Foundations

2.1 Image Semantic Segmentation

Semantic segmentation assigns a class label to each pixel in image, which allows detailed understanding of the scene. In comparison to image classification, which gives one label per image, it produces a map that shows object boundaries and categories. This is useful in tasks like autonomous driving and medical image analysis.

2.2 Pre-training and Fine-tuning

Pre-training includes training model on large dataset in order to learn and understand general features. Fine-tuning adapts these features to smaller and more task specific dataset. This improves performance and reduces training time in comparison to training the model from scratch.

3 Datasets and Experimental Setup

3.1 Datasets

We used two datasets: **OxfordIIITPet** and **Cityscapes**. OxfordIIITPet includes images of the pets and it is segmented in 3 classes: pet, border, and background. The Cityscapes dataset contains street scenes with 30 labels, but we only used 19 classes for training. Pixels which are labeled as 255 were ignored during training and evaluation. To match PyTorch's zero indexing, we subtracted label values by 1 in both datasets

3.2 Models

There are 2 main models. The first **FCN-ResNet50** is trained both from scratch and using pre-trained encoder. The second, **SegFormer**, is a transformer-based model which we implemented during the assignment. Additionally, we experimented with fine-tuning of the SegFormer, where we pretrained it on Cityscapes and trained on OxfordIIITPet.

3.3 Configuration

For all runs and models, we used AdamW optimizer, with asmgard=True. Batch size for the OxfordIIITPet was 64 and for Cityscapes was 16. Models are trained 30-40 epochs, depending on the experiment. Additionally, Exponential Learning Rate Scheduler was used to reduce learning rate over time and also Cross Entropy Loss was used for optimization. At the end, we had `ignore_index = 255` which is applied to unused classes in Cityscapes dataset. At the end, every second epoch we checked the best model and saved it for validation, with addition of the last model at the end.

4 Results and Evaluation

We ran several combinations of the models, where we combined if the model is pre-trained and also if the encodes was frozen or not. As stated, for FCN-ResNet50 we use pre-trained model from the official PyTorch Library (Trained on COCO dataset), while SegFormer was pre-trained on the Cityscapes dataset and used for OxfordIIITPet dataset. The overview of setting combinations is represented in table 1.

Model	Pre-trained On	Dataset	Encoder Frozen	LR
FCN-ResNet50	No	OxfordIIITPet	No	1×10^{-3}
FCN-ResNet50	Yes (ImageNet\COCO)	OxfordIIITPet	No	1×10^{-3}
SegFormer	No	OxfordIIITPet	No	1×10^{-3}
SegFormer	No	Cityscapes	No	1×10^{-3}
SegFormer	Yes (Cityscapes)	OxfordIIITPet	No	1×10^{-3}
SegFormer	Yes (Cityscapes)	OxfordIIITPet	No	5×10^{-4}
SegFormer	Yes (Cityscapes)	OxfordIIITPet	Yes	1×10^{-3}
SegFormer	Yes (Cityscapes)	OxfordIIITPet	Yes	5×10^{-4}

Table 1: Experiment configurations with dataset, pre-training, encoder freezing, and learning rate.

In addition of experiments execution, we tried to plot and track to important values: **Mean Intersection over Union (mIoU)** and Training\Validation Loss. The plots for training and validation are shown on the Figure 1.

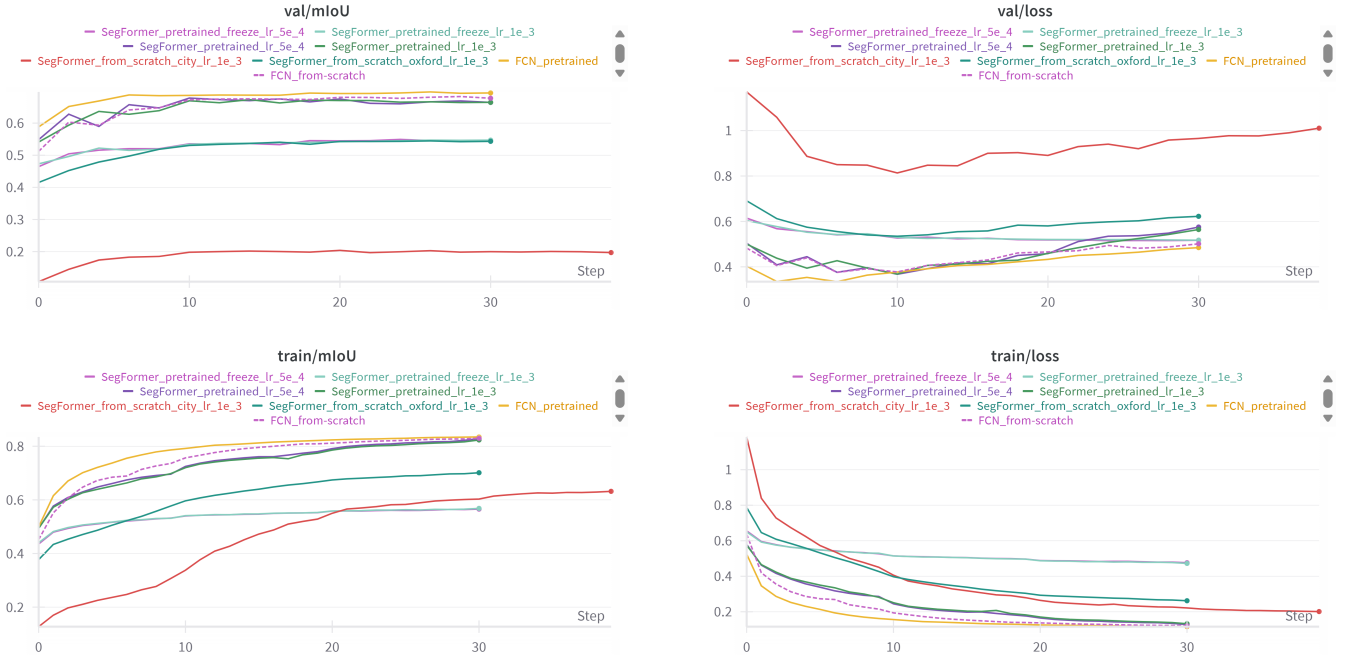


Figure 1: Training\Validation Loss and mIoU

4.1 Training

The FCN models showed the best training performance, where **FCN pretrained** achieved the highest **mIoU of 0.835** and the lowest training **loss of 0.116**. This might be because of strong and effective learning of features. Additionally, it has stable CNN-based architecture and well-trained backbone weights. In total, both FCN variants did well, which might be an indicator that ResNet50 is a network robust to initialization.

On the other hand, SegFormer models trained from scratch showed lower training **mIoU of 0.701** for OxfordIIITPet and **0.632** for Cityscapes. **SegFormer models with frozen pre-trained encoders** performed the worst in training, where **mIoU 0.565** and **loss 0.476** with **learning rate 5e-4**. This is probably because the encoder could not adapt to the OxfordIIITPet dataset, while the decoder had limited ability to compensate by itself. All other combinations of SegFormer showed almost the same training mIoU as FCN ResNet50 models.

4.2 Validation

In validation, FCN pretrained showed again the best **mIoU at 0.694** and lowest **loss of 0.485**. That confirms it is not only trained well but also effectively generalizes unknown data. The **FCN scratch** version followed with **0.677 of mIoU**, which shows that even without pretraining, the FCN performs well on OxfordIIITPet. It is possibly because of the simpler task structure, consisting only of 3 classes and well-aligned spatial features.

For SegFormer, the versions with encoder pretraining, with **learning rates of 1e-3 and 5e-4** achieved around **mIoU of 0.665-0.664**, showing good generalization but it still underperforms compared to FCN. It is interesting that **freezing the encoder** extremely decreased performance to **0.545–0.547 mIoU**. This suggests that freezing the encoder does not allow the model to adapt to critical features for OxfordIIITPet dataset (e.g., pet contours or textures), even though overfitting is somehow reduced.

The worst validation result came from the SegFormer trained on **Cityscapes only**, with just mIoU around 0.200. This confirms that generalization without fine-tuning is poor, especially when label sets and their characteristics (urban scenes vs. pets) are very different. More detailed results are shown in the Table 2.

Model	LR	Train mIoU	Val mIoU	Train Loss	Val Loss
FCN pretrained	1×10^{-3}	0.835	0.694	0.116	0.485
FCN from-scratch	1×10^{-3}	0.830	0.677	0.121	0.501
SegFormer pretrained	1×10^{-3}	0.823	0.665	0.133	0.564
SegFormer pretrained	5×10^{-4}	0.829	0.664	0.128	0.575
SegFormer pretrained freeze	1×10^{-3}	0.568	0.547	0.472	0.517
SegFormer pretrained freeze	5×10^{-4}	0.565	0.545	0.476	0.517
SegFormer from scratch oxford	1×10^{-3}	0.701	0.543	0.262	0.622
SegFormer from scratch city	1×10^{-3}	0.632	0.200	0.221	1.011

Table 2: Training and validation mIoU and loss for all model configurations, with corresponding learning rates.

At the end, we show the images of the OxfordIIITPet datasets which are produced by the SegFormer model. The first image are masks made by model from scratch, the second one by fine-tuning and the third one by freezing the pre-trained model.

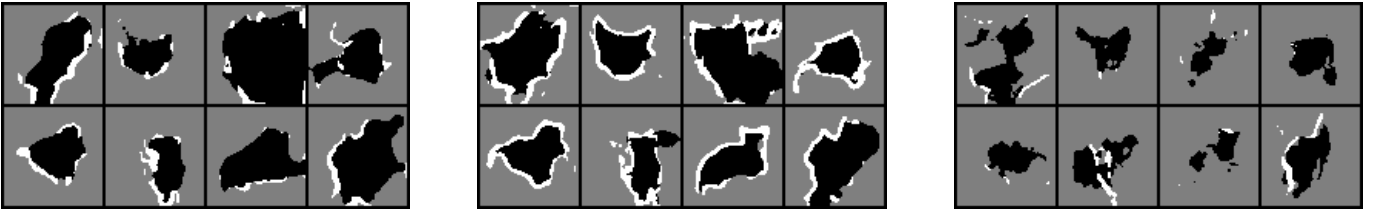


Figure 2: Segmentation masks example: scratch, fine-tuning, and freeze.

5 Conclusion and Discussion

In this assignment, we explored image segmentation using two architectures: FCN-ResNet50 and SegFormer. Our experiments on OxfordIIITPet dataset showed that **FCN-ResNet50 with a pre-trained weights** achieved the best performance in general, with the highest validation mIoU and lowest loss. This shows the strong generalization ability of CNN-based architectures when they are combined with weight initialization.

While SegFormer models provided close results, especially when they are fine-tuned from pre-trained Cityscapes encoders, they did not performed better than the simpler FCN approach. We can notice that freezing the encoder during fine-tuning gives significant drops in performance, which highlights how the important is to allow model to adapt to new domains and datasets. Pre-training on non-related datasets like Cityscapes without fine-tuning did not provide good generalization, which again highlights the importance of task-specific adaptation.

In general, these results show that both model architecture and training strategy, which includes pre-training, fine-tuning, and learning rate, have critical roles in getting the strong segmentation performance. For simpler tasks, CNN models with pre-trained backbones still provides great performance, while transformer-based models needs more careful and precise tuning to reach their full potential.

5.1 Learning rate

We also tried to evaluate the effect of different learning rates during fine-tuning. We tried to use 2 learning rates: $5e^4$ and $1e^3$. SegFormer pre-trained models without freeze showed more or less similar performance for both values of

learning rates (0.665 and 0.664, respecting learning rates values). However, if we include also freeze, than the overall validation mIoU decreases, but again there is no significant gap between the results using different learning rates (0.545 and 0.547 respecting learning rates values). This suggests that, in our case, learning rate had a minor impact on final performance, when they use the same overall training setup.

5.2 Answers to the questions

1. What other architectures for segmentation apart from the SegFormer have you learned in the lecture and how do they differ from the SegFormer?

In the lecture, we studied about **U-Net** and **Swin Transformer**. **U-Net** is architecture which is based on CNN encoder-decoder, where skipping connections helps to save details in a way of combining low-level and high-level features during up-sampling. On the other hand, **Swin Transformer** uses hierarchical vision transformers with shifted windows to model dependencies with keeping the computational efficiency. Compared to SegFormer, which also uses transformer-based encoder, Swin Transformers uses positional encoding and complex self-attention mechanisms, while SegFormer focuses on simplicity and faster scalability but it does not include positional encoding.

2. Why are those architectures using down- and up-sampling instead of keeping the same resolution?

Downsampling reduces the resolution, which allows the model to extract high-level features and increase receptive field and all of that with reduction of computational cost. Up-sampling returns images to the original resolution in order to produce pixel-level predictions. The reason behind using up- and down- sampling is to **reduce the computational costs and memory usage**.

3. What is the purpose of pre-training and fine-tuning? For which use-cases is it especially beneficial?

Pre-training allows model to learn general features on large dataset, which can then be used for the specific task with applying of fine-tuning. This is especially beneficial when the specific dataset is small or diversity of the data is low. It helps to improve generalization, speed up convergence, and reduce the risk of overfitting, because using the complex architectures on small datasets can lead to those issues.

4. What have you observed in your experiments above, how has using pre-trained weights influenced the performance and why? Do you see any differences between the two fine-tuning options of part 6?

Using pre-trained weights improved performance over training from scratch. The **SegFormer with a fine-tuned pre-trained encoder** showed better validation mIoU than model trained from scratch and frozen encoder variant. Freezing the encoder provided significantly lower results, which indicates that the model needs to adapt feature representations to the new domain and datasets. We also compared learning rates (1×10^{-3} vs. 5×10^{-4}) and we found only small differences in results, which gave us an idea that the learning rate had limited effect for this training setup, although it is the good recommendation to use lower learning rates for fine-tuning.